

深圳大学实验报告

课程名称： 算法设计与分析

实验项目名称： AI 推荐系统中的算法效率分析及优化

学院： 计算机与软件学院

专业： 计算机科学与技术

指导教师： 刘刚

报告人： 汉雨 学号： 2024040042

实验时间： 2026 年 3 月 21 日—2026 年 4 月 4 日

实验报告提交时间： 2026 年 4 月 3 日

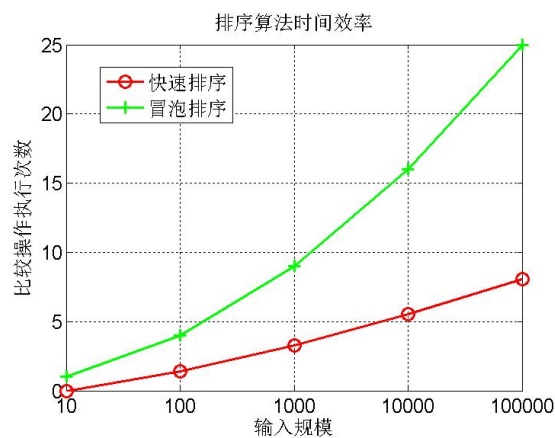
教务部制

一、实验目的：

1. 掌握选择排序、冒泡排序、插入排序、合并排序、快速排序算法原理
2. 掌握不同排序算法时间效率的经验分析方法，验证理论分析与经验分析的一致性。
3. 求解 TOP K 问题，并分析比较不同算法效率。

二、实验内容：

- 1、实现选择排序、冒泡排序、插入排序、合并排序、快速排序算法；
- 2、以待排序数组的大小 n 为输入规模，固定 n ，随机产生 20 组测试样本，统计不同排序算法在 20 个样本上的平均运行时间；
- 3、分别以 $n=10$ 万, $n=20$ 万, $n=30$ 万, $n=40$ 万, $n=50$ 万等等（输入规模 n 可以直至极限），重复 2 的实验，画出不同排序算法在 20 个随机样本的平均运行时间与输入规模 n 的关系，如下图 1 所示，注意横坐标要均匀。



图表 1

图 1. 时间效率与输入规模 n 的关系图

4、画出理论效率分析的曲线和实测的效率曲线，注意：由于实测效率是运行时间，而理论效率是基本操作的执行次数，两者需要进行对应关系调整。调整思路举例：以输入规模为 10 万的数据运行时间为基准点，计算输入规模为其他值的理论运行时间，画出不同规模数据的理论运行时间曲线，并与实测的效率曲线进行比较。经验分析与理论分析是否一致？如果不一致，请解释存在的原因。

5、现在有 10 亿的数据（每个数据四个字节），请快速挑选出最大的十个数，并在小规模数据上验证算法的正确性。

一、 问题描述

首先，基本问题是对视频、音乐或电商平台中的用户进行内容推荐。将使用某种排序算法对规模为 n 的用户兴趣偏好的余弦相似度进行排序，并找出相似度为前 K 的用户（TOP-K），以此根据这些用户的偏好进行推荐。

随后会以冒泡排序、选择排序、插入排序、合并排序、快速排序、堆排序等基础排序算法为基础，分析各个算法的基本原理、复杂度、理论效率与实测时间的关系、不同用户规模下各类排序算法的表现、不同 K 值下算法的表现。最后比较各个算法的效率，综合优化算法。

二、 算法设计

数据生成

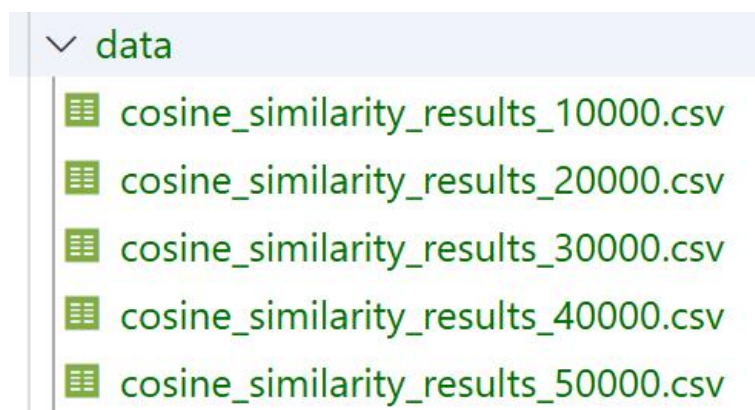
在本实验中，为了系统分析不同排序算法在不同规模数据下的性能表现，数据生成过程设计如下：

首先，按照不同数量级构造实验数据，分别生成了 $n = 10000, 20000, \dots, 100000$ 五种规模的数据集。通过设置不同规模，可以观察算法在不同规模输入下的时间复杂度变化趋势，从而更直观地比较各算法的效率差异。

其次，为保证实验的公平性与可比性，所有生成的数据均统一存储在 `workspace/hanyu_workspace/data` 文件夹中。不同排序算法在测试时均使用同一批数据，避免由于输入数据差异对实验结果产生干扰，从而实现严格的控制变量。

在数据精度方面，本实验中生成的相似度数据统一保留小数点后两位。这一设置既能降低存储与计算开销，又能满足实验分析的基本需求。。

最后，为便于数据管理与后续处理，不同规模的数据分别写入以 `cosine_similarity_results_xxx.csv` 命名的 CSV 文件中（其中 `xxx` 表示数据规模）。这种方式使得各算法可以直接读取对应文件进行处理，提高了实验的可复现性与执行效率。



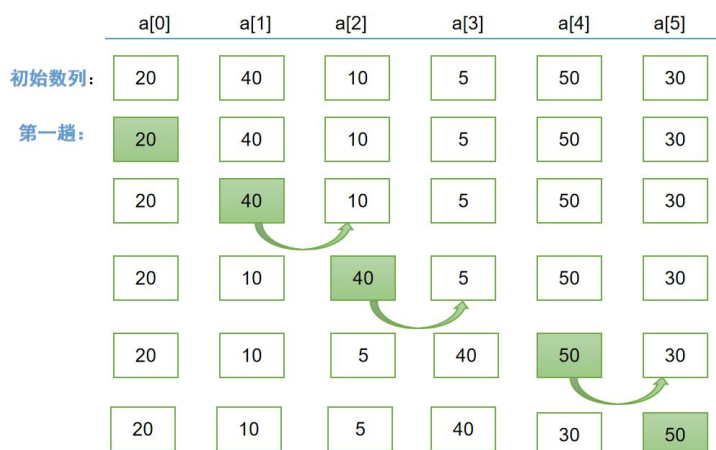
图表 1

冒泡排序算法分析

- 冒泡排序算法描述

从序列的第一个元素开始，比较相邻的两个元素，如果它们的顺序错误（如：从大到小排序时，前一个元素大于后一个元素），则交换它们的位置。继续比较下一对相邻元素，执行相同的操作，直到序列的末尾。

若以升序（从小到大）排序为例，假若有 N 个数。第一趟排序目的是找到整个数组中最大的数并把它排在最后端；最后一个最大数不再比较移动，第二趟排序目的是在剩下的 $N-1$ 个数找出最大的（即整个数组中第二大的数）并把它放在倒数第二位……这样一轮一轮的比较，直到只剩下一个数时（完成了 N 趟的排序）这个排序就完成了，从而实现从小到大的排序。



图表 2 冒泡排序过程

- 冒泡排序伪代码

Input: Array $\mathbf{a}$ consisting of $\mathbf{n}$ elements

Output: $\mathbf{a}$ will be sorted in nonincreasing order

```
function BubbleSort(a, n)
  for i ← 1 to n - 1 do
    for j ← 1 to n - i do
      if a[j] < a[j + 1] then
        swap a[j] and a[j + 1]
      end if
    end for
  end for
end function
```

- 冒泡排序实现 TOP-k

冒泡排序的基本思想是：每一趟比较相邻两个元素，把较大（或较小）的元素逐步“冒”到一端。假设有 n 个元素：第 1 趟需要比较 $n-1$ 次，第 2 趟需要比较 $n-2$ 次，第 3 趟需要比较 $n-3$ 次 ... 第 $n-1$ 趟需要比较 1 次，所以总比较次数为：

$$(n-1) + (n-2) + \cdots + 1$$

这是一个等差数列求和：

$$\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2}$$

因此其时间复杂度为：

$$O(n^2)$$

如果数组本身已经有序，并且设置了“本趟无交换则提前结束”的标志，那么只需要扫描一趟，即：

$$O(n)$$

冒泡排序需要对全部相似度值进行排序，再取前 k 个。

排序复杂度为：

$$T_{\text{sort}}(n) = O(n^2)$$

取前 k 个的复杂度为：

$$O(k)$$

因此总复杂度为：

$$T(n, k) = O(nd + n^2 + k)$$

若忽略低阶项，可写为：

$$T(n, k) = O(n^2)$$

● 冒泡排序实验结果与分析

本实验采用了不同规模的数据作为输入，并统一保存在数据文件中，以保证各算法测试条件一致。我们通过编写 C++ 脚本测量了算法在不同数据规模下的实际运行时间，同时根据时间复杂度公式推导出理论用时。最后，将实验结果整理成一张表格 [/workspace/hanyu_workspace/result/bubble_compare_results.csv](#)，并绘制了实际时间与理论时间的对比曲线图 [/workspace/hanyu_workspace/result/bubble_compare.png](#)，用于分析算法理论效率与实际表现之间的关系。

表 1 冒泡排序实验结果

n	actual_time_ms	theoretical_time_ms
10000	128.887	128.887
20000	515.876	515.549
30000	1157.76	1159.99
40000	2075.17	2062.2
50000	3302.91	3222.18
60000	4876.45	4639.94
70000	7198.64	6315.48
80000	10045.6	8248.79
90000	13407.4	10439.9
100000	17279.3	12888.7

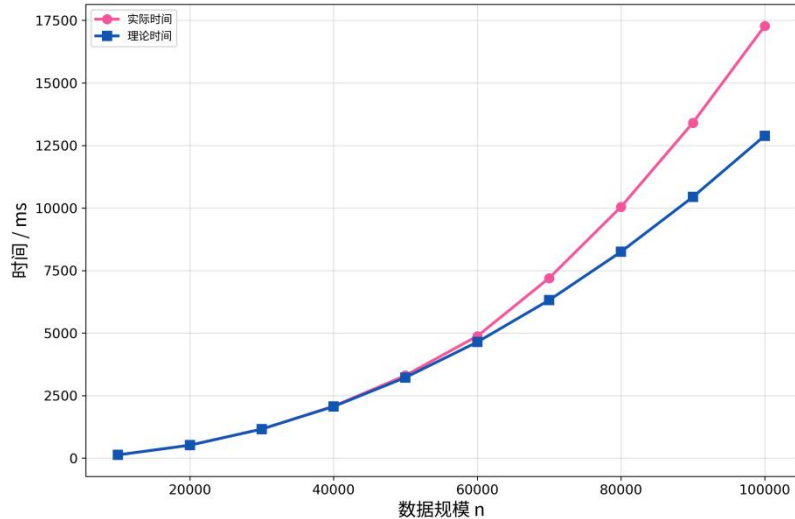


图 3 冒泡排序理论时间与实际时间对比

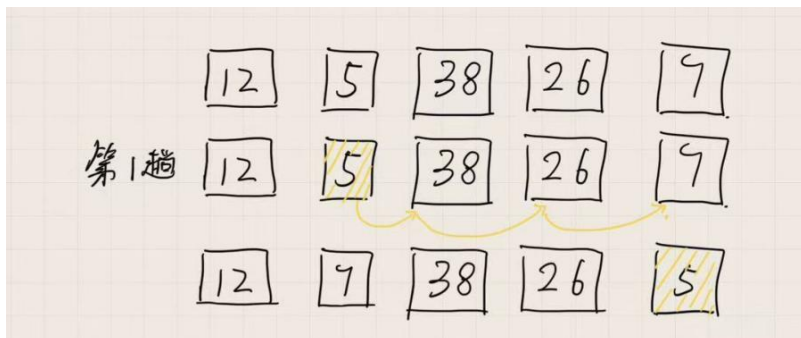
从实验结果可以看出，冒泡排序的实际运行时间总体上随着数据规模增大呈二次增长趋势，与理论时间复杂度 $O(n^2)$ 基本一致。但理论时间曲线与实际时间曲线并不完全重合，且在数据规模较大时偏差逐渐增大。这是因为理论分析只保留了主导项，忽略了常数项和低阶项，同时实际运行时间还会受到编译器优化、缓存机制、系统环境以及数据分布等因素影响。因此，理论分析能够正确反映算法增长趋势，但不能完全等同于程序在真实环境中的运行时间

选择排序算法分析

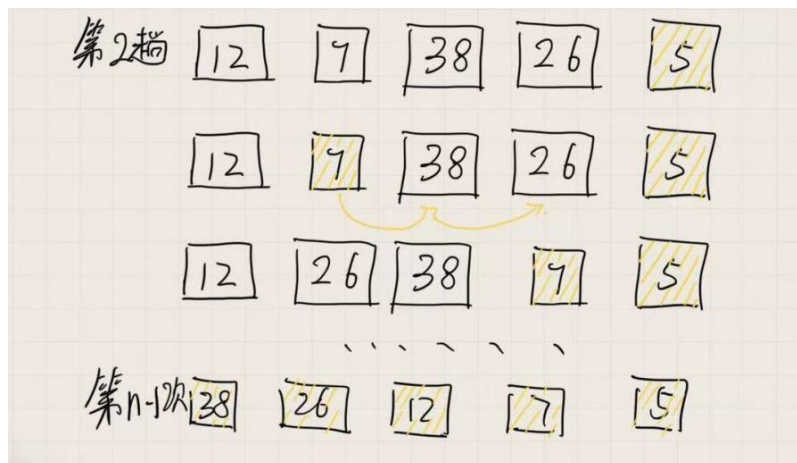
● 选择排序算法描述

选择排序的基本思想是：在待排序序列中，每一趟从未排序部分中选出一个最小（或最大）的元素，然后将其放到当前应在的位置上。这样经过多趟比较之后，整个序列就能够按照要求排好序。

如果按升序排列，具体过程如下：在第 1 趟排序中，从全部元素中选出最小的元素，与第 1 个位置上的元素交换；在第 2 趟排序中，从剩余未排序的元素中选出最小的元素，与第 2 个位置上的元素交换；依次重复上述过程；当未排序部分只剩下 1 个元素时，排序完成。



图表 4 选择排序过程



图表 5 选择排序过程（续）

- 选择排序伪代码

Algorithm : **SelectSort**

Input: Array **a** consisting of **n** elements

Output: **a** will be sorted in nondecreasing order.

```
function SelectSort(a, n)
  for i ← 1 to n - 1 do
    min ← i
    for j ← i + 1 to n do
      if a[min] > a[j] then
        min ← j
      end if
    end for
    if min ≠ i then
      swap a[min] and a[i]
    end if
  end for
end function
```

- 选择排序实现 TOP-k

选择排序每一趟从未排序区间中找出最小值（或最大值），再放到正确位置。

假设有 n 个元素：第 1 趟要在 n 个元素中找最小值，需要比较 $n-1$ 次，第 2 趟要在剩下 $n-1$ 个元素中找最小值，需要比较 $n-2$ 次 ... 第 $n-1$ 趟需要比较 1 次
因此总比较次数也是：

$$(n-1) + (n-2) + \dots + 1$$

即：

$$\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2}$$

所以选择排序时间复杂度为：

$$O(n^2)$$

选择排序同样需要完整排序后再取前 k 个，排序复杂度为：

$$T_{\text{sort}}(n) = O(n^2)$$

总复杂度为：

$$T(n, k) = O(nd + n^2 + k)$$

简化后：

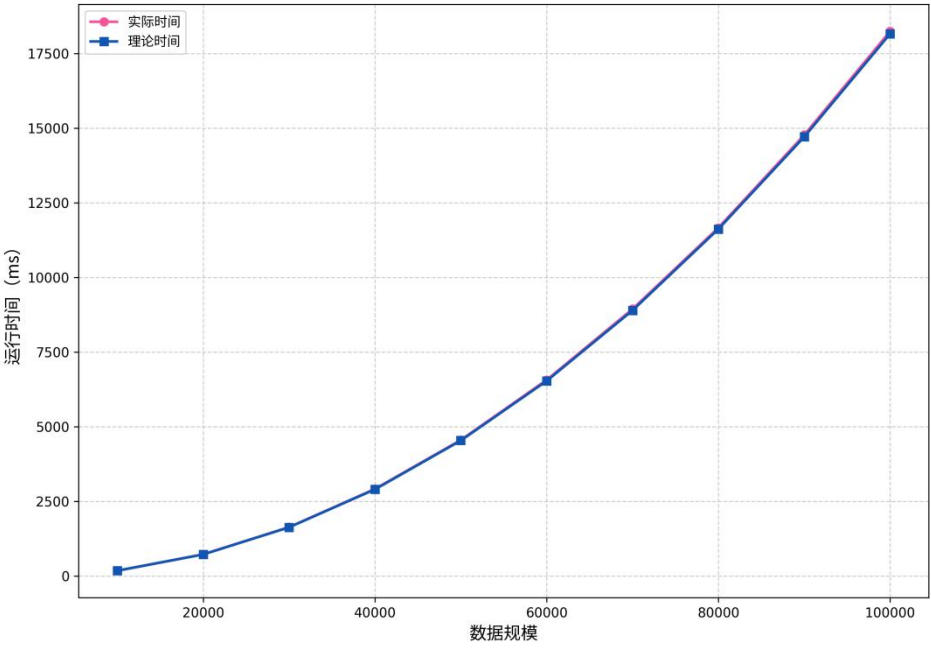
$$T(n, k) = O(n^2)$$

● 选择排序实验结果

本实验采用了不同规模的数据作为输入，并统一保存在数据文件中，以保证各算法测试条件一致。我们通过编写 C++ 脚本测量了算法在不同数据规模下的实际运行时间，同时根据时间复杂度公式推导出理论用时。最后，将实验结果整理成一张表格 [/workspace/hanyu_workspace/result/select_compare_results.csv](#)，并绘制了实际时间与理论时间的对比曲线图 [/workspace/hanyu_workspace/result/select_compare.png](#)，用于分析算法理论效率与实际表现之间的关系。

表 2 选择排序实验结果

n	actual_time_ms	theoretical_time_ms
10000	181.586100	181.586100
20000	728.904096	726.344401
30000	1640.378419	1634.274901
40000	2918.311384	2905.377602
50000	4556.914555	4539.652504
60000	6569.152569	6537.099605
70000	8942.757910	8897.718907
80000	11664.574335	11621.510410
90000	14772.450699	14708.474112
100000	18243.123299	18158.610015



图表 6 选择排序理论时间与实际时间对比

虽然冒泡排序和选择排序的理论时间复杂度同为 $O(n^2)$ ，但实验结果中两者与理论曲线的贴合程度并不完全相同。选择排序的实际时间与理论时间更接近，这是因为选择

排序每一趟的比较次数基本固定，交换次数较少，算法执行过程较为稳定，因此更符合理论模型的假设。相比之下，冒泡排序在执行过程中除了比较之外还伴随大量交换操作，并且其运行时间更容易受到数据分布、实现细节以及是否提前结束等因素影响，因此实际时间与理论时间之间会出现更明显的偏差。

插入排序算法分析

- 插入排序算法描述

插入排序是一种比较直观、简单的排序算法，它的思想和人们整理扑克牌时很相似。算法将序列分为“已排序部分”和“未排序部分”，开始时默认第一个元素已经有序，然后依次从未排序部分取出一个元素，将它与前面已排序部分的元素从后往前比较，找到合适的位置后插入进去。这样经过多轮操作后，整个序列就会变成有序。插入排序在数据量较小时效率较高，而且如果原始序列本身已经基本有序，它的表现会更好。它的最好时间复杂度是 $O(n)$ ，平均和最坏时间复杂度都是 $O(n^2)$ ，空间复杂度为 $O(1)$ ，属于原地排序。同时，插入排序是一种稳定排序，也就是说相同元素的相对次序不会改变。它的优点是实现简单、容易理解，适合小规模数据排序；缺点是当数据规模较大时，比较和移动次数会明显增多，效率不如快速排序、堆排序等更高效的算法。



图表 7 选择排序过程

- 插入排序伪代码

Algorithm InsertSort

Input: Array a consisting of n elements

Output: a will be sorted in nondecreasing order.

```
function INSERTSORT( $a, n$ )
  for  $i \leftarrow 1$  to  $n - 1$  do
     $key \leftarrow a[i]$ 
     $j \leftarrow i - 1$ 
    while  $j > 0$  and  $a[j] > key$  do
       $a[j + 1] \leftarrow a[j]$ 
       $j \leftarrow j - 1$ 
    end while
     $a[j + 1] \leftarrow key$ 
  end for
end function
```

- 插入排序实现 TOP-k

插入排序的思想是：把当前元素插入到前面已经有序的序列中。最坏情况是原序列逆序。这时第 i 个元素插入时，需要和前面所有元素比较并移动。第 2 个元素，最多比较 1 次，第 3 个元素，最多比较 2 次 ... 第 n 个元素，最多比较 $n - 1$ 次。总比较/移动次数为：

$$1 + 2 + \cdots + (n - 1)$$

即：

$$\sum_{i=1}^{n-1} i = \frac{n(n-1)}{2}$$

所以最坏情况复杂度为：

$$O(n^2)$$

最好情况是原序列已经有序。

这时每次插入只需要比较一次，不需要大量移动，因此总操作次数近似为： $n-1$ 所以最好情况复杂度为：

$$O(n)$$

综合得：最好情况： $O(n)$ ；平均情况： $O(n^2)$ ；最坏情况： $O(n^2)$ 。插入排序在平均和最坏情况下复杂度为：

$$T_{\text{insertion-sort}}(n) = O(n^2)$$

总复杂度：

$$T_{\text{insertion}}(n, k) = O(n^2 + k)$$

简写为：

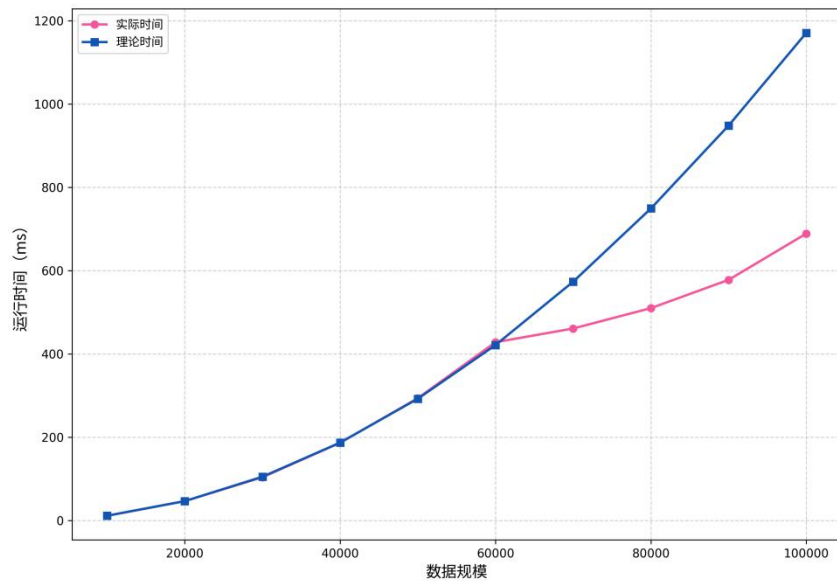
$$T_{\text{insertion}}(n, k) = O(n^2)$$

- 插入排序实验结果

本实验采用了不同规模的数据作为输入，并统一保存在数据文件中，以保证各算法测试条件一致。我们通过编写 C++ 脚本测量了算法在不同数据规模下的实际运行时间，同时根据时间复杂度公式推导出理论用时。最后，将实验结果整理成一张表格

/workspace/hanyu_workspace/result/insert_compare_results.csv，并绘制了实际时间与理论时间的对比曲线图/workspace/hanyu_workspace/result/insert_compare.png，用于分析算法

理论效率与实际表现之间的关系。

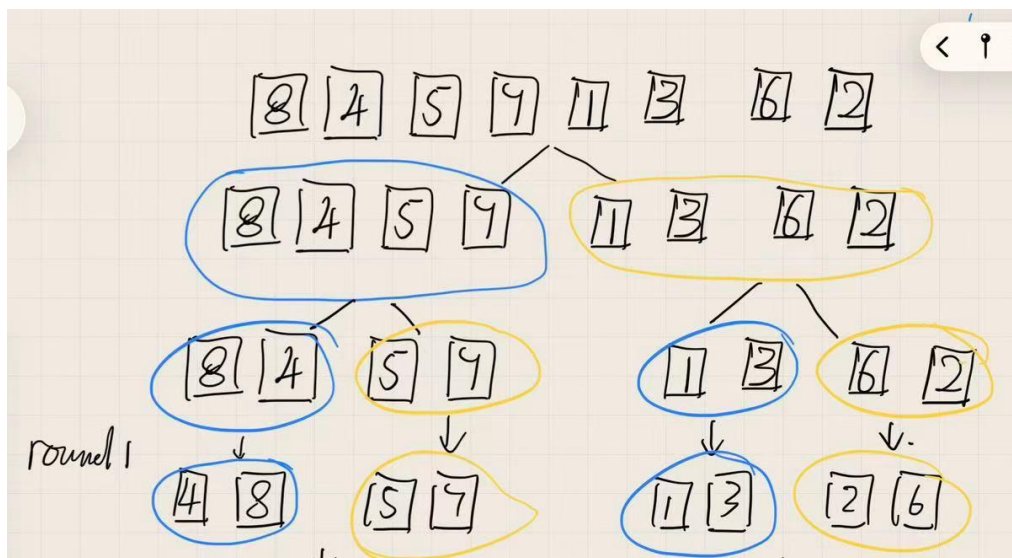


图表 8 插入排序理论时间与实际时间对比

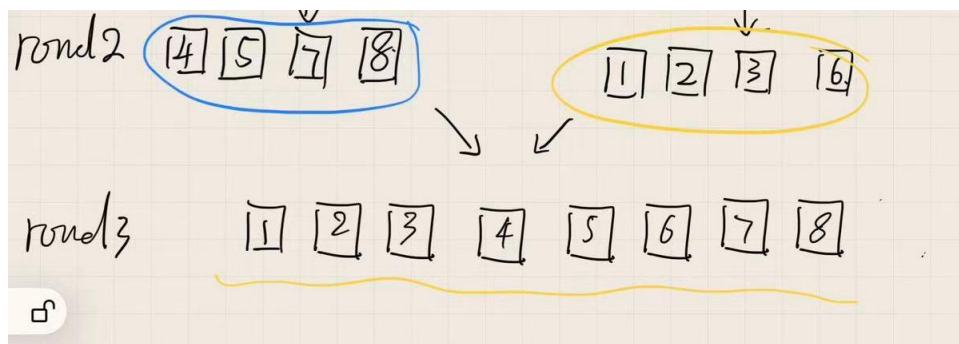
合并排序算法分析

● 合并排序算法描述

合并排序，也叫归并排序，是一种基于**分治思想**的排序算法。它的基本思路是：先把一个大的无序序列不断地**分成两个较小的子序列**，直到每个子序列中只剩下一个元素为止，因为单个元素本身就是有序的；然后再按照大小关系将这些已经有序的子序列**两两合并**，逐步合成为更大的有序序列，最终得到整个序列的排序结果。合并排序的时间复杂度比较稳定，在最好、平均和最坏情况下都为 $O(n \log n)$ ，这也是它的重要优点之一。它的空间复杂度是 $O(n)$ ，因为在合并过程中通常需要额外的辅助数组来存放临时结果，所以它不是原地排序。另一方面，合并排序属于**稳定排序**，也就是说相同元素排序后相对位置不会改变。总体来说，合并排序适合处理规模较大的数据，尤其在需要稳定排序时很有优势，但缺点是需要额外的存储空间，实现上也比插入排序、选择排序稍复杂一些。



图表 9 合并排序过程



图表 10 合并排序过程（续）

- 合并排序伪代码

Algorithm MergeSort

Input: Array a with left boundary $left$ and right boundary $right$

Output: a will be sorted in non-decreasing order between $left$ and $right$.

```
function MERGESORT( $a$ ,  $left$ ,  $right$ ,  $temp$ )
```

```
    if  $left \geq right$  then
```

```
        return
```

```
    end if
```

```
     $mid \leftarrow left + right \gg 1$ 
```

```
    MERGESORT( $a$ ,  $left$ ,  $mid$ ,  $temp$ )
```

```
    MERGESORT( $a$ ,  $mid + 1$ ,  $right$ ,  $temp$ )
```

```
     $idx \leftarrow 0$ 
```

```
     $i \leftarrow left$ 
```

```
     $j \leftarrow mid + 1$ 
```

```
    while  $i \leq mid$  and  $j \leq right$  do
```

```
        if  $a[i] < a[j]$  then
```

```
             $temp[idx++] \leftarrow a[i++]$ 
```

```
        else
```

```
             $temp[idx++] \leftarrow a[j++]$ 
```

```
        end if
```

```
    end while
```

```
    while  $i \leq mid$  do
```

```
         $temp[idx++] \leftarrow a[i++]$ 
```

```
    end while
```

```
    while  $j \leq right$  do
```

```
         $temp[idx++] \leftarrow a[j++]$ 
```

```
    end while
```

```
     $j \leftarrow 0$ 
```

```
    for  $i \leftarrow left$  to  $right$  do
```

```

        a[i] ← temp[j++]
    end for
end function

```

● 合并排序实现 TOP-k

合并排序采用“分治”思想：把序列不断分成两半分别排序，再将两个有序子序列合并设其时间复杂度为 $T(n)$ ，则有递推式：

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

其中：

- $2T(n/2)$ ：表示递归处理左右两个子问题
- $O(n)$ ：表示合并两个有序子序列所需时间

因为不断二分，所以递归层数约为：

$$\log_2 n$$

而每一层合并操作的总代价都是：

$$O(n)$$

因此总复杂度为：

$$O(n \log n)$$

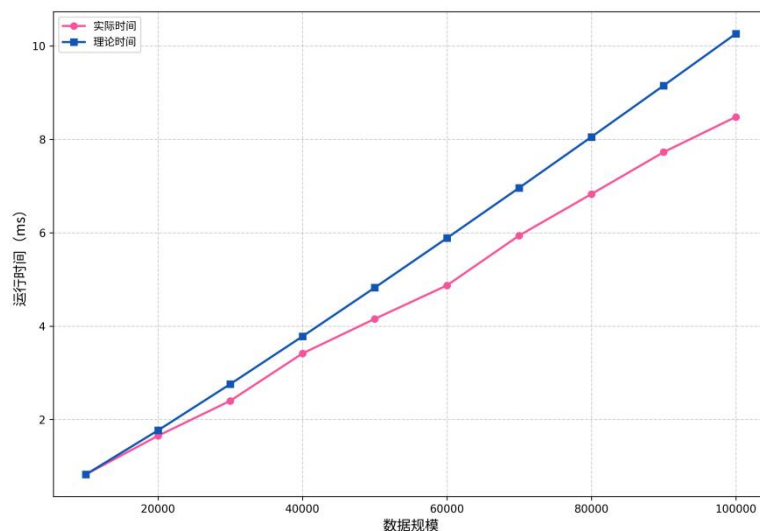
● 合并排序实验结果

本实验采用了不同规模的数据作为输入，并统一保存在数据文件中，以保证各算法测试条件一致。我们通过编写 C++ 脚本测量了算法在不同数据规模下的实际运行时间，同时根据时间复杂度公式推导出理论用时。最后，将实验结果整理成一张表格

[/workspace/hanyu_workspace/result/merge_compare_results.csv](#)，并绘制了实际时间与理论时间的对比曲线图[/workspace/hanyu_workspace/result/merge_compare.png](#)，用于分析算理论效率与实际表现之间的关系。

表 3 合并排序实验结果

n	actual_time_ms	theoretical_time_ms
10000	0.821115	0.821115
20000	1.649636	1.765820
30000	2.398503	2.757174
40000	3.413435	3.778820
50000	4.154131	4.822993
60000	4.870988	5.885117
70000	5.940104	6.962170
80000	6.828068	8.052001
90000	7.726352	9.153006
100000	8.481255	10.263937



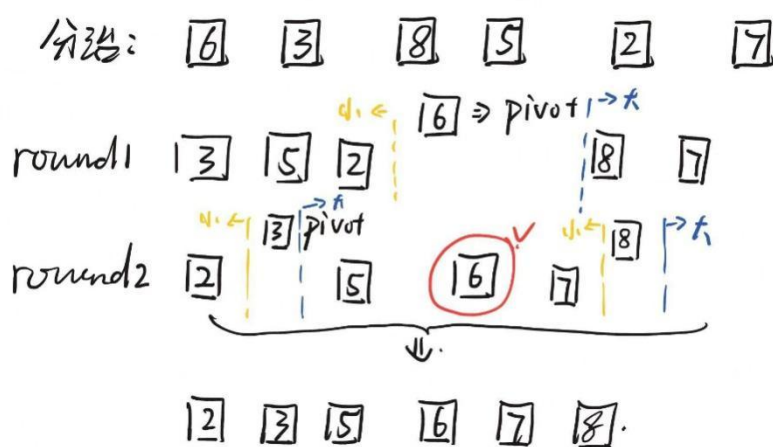
图表 11 合并排序理论与实际时间对比

从实验结果来看，合并排序的实际运行时间随着输入规模增大而稳定增长，其变化趋势与理论时间复杂度 $O(n \log n)$ 基本一致，说明经验分析与理论分析总体相符。但从具体数值上看，实际时间与理论时间并不完全相同，且在数据规模较大时理论时间略高于实际时间。这主要是因为理论复杂度只反映算法的渐进增长规律，忽略了常数项和低阶项；同时，实际运行时间还会受到编译器优化、内存访问、系统环境以及计时误差等多种因素影响，因此两者之间出现一定偏差是正常的。

快速排序算法分析

● 快速排序算法描述

快速排序是一种基于分治思想的排序算法，其基本做法是先在待排序序列中选取一个元素作为基准，然后通过一趟划分操作，将所有小于基准的元素放到基准左侧，将所有大于基准的元素放到基准右侧，从而使基准元素处于最终排序后应在的位置，接着再对基准左右两部分子序列分别递归地进行同样的排序处理，直到各个子序列的长度为 1 或 0 时排序结束，最终得到整体有序序列。快速排序的优点是平均情况下排序效率较高。



图表 12 快速排序过程

- 快速排序伪代码

Algorithm : QuickSort

Input: Array a with left boundary low and right boundary $high$

Output: a will be sorted in non-decreasing order between low and $high$.

```
function QUICKSORT( $a$ ,  $low$ ,  $high$ )
    if  $low \geq high$  then
        return
    end if

     $l \leftarrow low - 1$ 
     $r \leftarrow high + 1$ 
     $pivot \leftarrow a[low + high \gg 1]$ 

    while  $l < r$  do

        repeat
             $l++$ 
        until  $a[l] \geq pivot$ 

        repeat
             $r--$ 
        until  $a[r] \leq pivot$ 

        if  $l < r$  then
            swap  $a[l]$  and  $a[r]$ 
        end if
    end while

    QUICKSORT( $a$ ,  $low$ ,  $r$ )
    QUICKSORT( $a$ ,  $r + 1$ ,  $high$ )
end function
```

- 快速排序实现 TOP-k

快速排序的思想是：选取一个枢轴（pivot），将序列划分成左右两部分，对左右部分递归排序 **平均情况**：若每次划分都比较均匀，则递推式为：

$$T(n) = 2T\left(\frac{n}{2}\right) + O(n)$$

和合并排序类似，每层划分代价为 $O(n)$ ，递归深度为 $O(\log n)$ ，所以平均复杂度为：

$$O(n \log n)$$

最坏情况：如果每次选到的枢轴都很差，比如每次都把数组划分成：一边 1 个元素，一边 $n - 1$ 个元素，则递推式变为：

$$T(n) = T(n - 1) + O(n)$$

展开后得到：

$$O(n + (n - 1) + (n - 2) + \cdots + 1) = O(n^2)$$

所以快速排序复杂度为：最好情况： $O(n \log n)$ ，平均情况： $O(n \log n)$ ，最坏情况： $O(n^2)$ ，快速排序平均复杂度为： $T_{\text{quick-sort,avg}}(n) = O(n \log n)$ 最坏复杂度为： $T_{\text{quick-sort,worst}}(n) = O(n^2)$

平均情况：

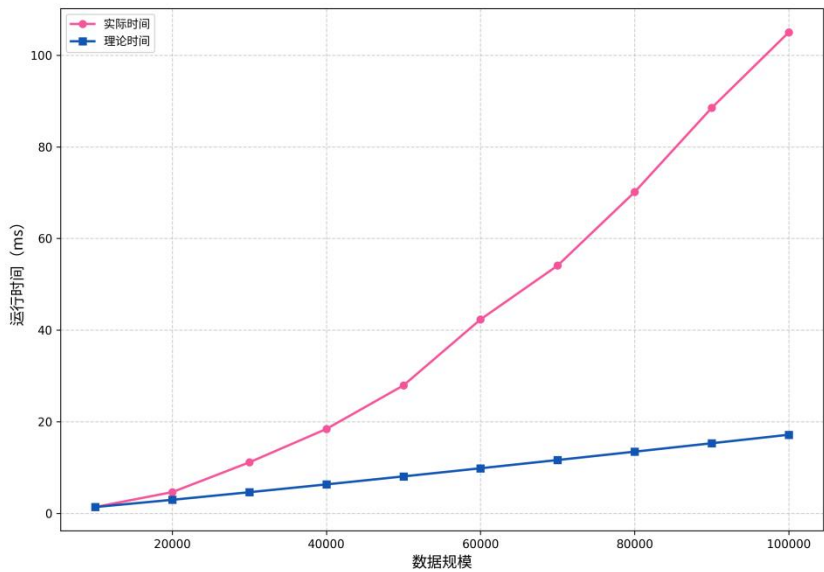
$$T_{\text{quick,avg}}(n, k) = O(n \log n + k)$$

● 快速排序实验结果

本实验采用了不同规模的数据作为输入，并统一保存在数据文件中，以保证各算法测试条件一致。我们通过编写 C++ 脚本测量了算法在不同数据规模下的实际运行时间，同时根据时间复杂度公式推导出理论用时。最后，将实验结果整理成一张表格 [/workspace/hanyu_workspace/result/quick_compare_results.csv](#)，并绘制了实际时间与理论时间的对比曲线图 [/workspace/hanyu_workspace/result/quick_compare.png](#)，用于分析算法理论效率与实际表现之间的关系。

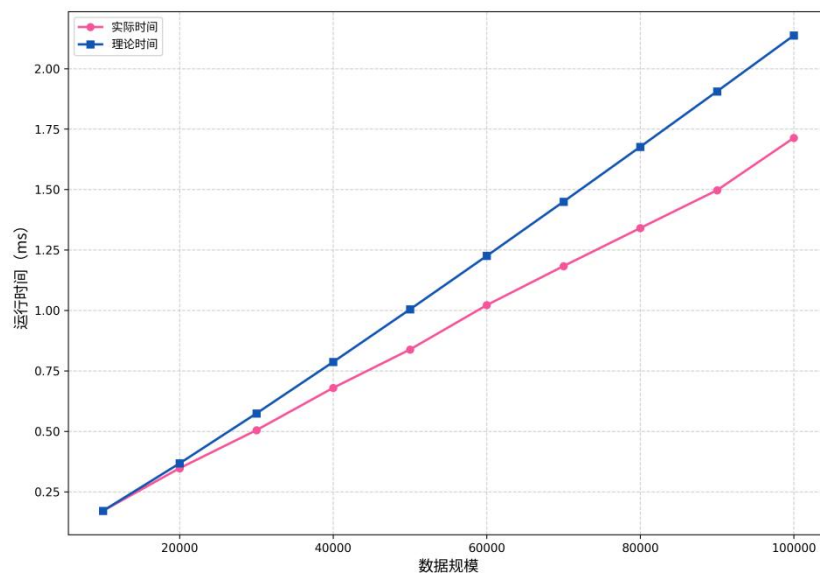
表 4 快速排序实验结果

n	actual_time_ms	theoretical_time_ms
10000	1.372298	1.372298
20000	4.628608	2.951146
30000	11.129353	4.607957
40000	18.418725	6.315396
50000	27.924787	8.060481
60000	42.324022	9.835568
70000	54.076894	11.635603
80000	70.122137	13.456997
90000	88.505728	15.297063
100000	104.994113	17.153719



图表 13 快速排序理论时间与实际时间对比

在快速排序的优化过程中，我们针对原始实现中划分不均衡的问题进行了改进。原始快速排序采用固定基准值并使用普通二路划分，当数据中存在大量重复元素时，与基准值相等的元素会集中到同一侧，容易导致递归子区间规模严重失衡，从而使实际运行效率下降。针对这一问题，我们将原算法改为**随机基准值 + 三路划分**的优化方案。其中，随机选取基准值可以降低固定位置选取基准所带来的退化风险；三路划分则将序列划分为“大于基准值”“等于基准值”和“小于基准值”三个部分，使得与基准值相等的元素能够一次性归入中间区域，不再参与后续递归。这样可以有效减少重复元素较多时的不平衡划分现象，提高快速排序在实际数据集上的运行效率，使算法性能更加稳定。



图表 14 三路快速排序理论时间与实际时间对比

堆排序算法分析

● 堆排序算法描述

堆排序是一种利用**堆这种数据结构**实现的排序算法，它的核心思想是：**先把序列构成一个堆，再不断取出堆顶元素完成排序**。如果要进行升序排序，通常构建的是**大顶堆**，也就是每个父节点的值都不小于它的子节点，这样堆顶元素就是当前序列中的最大值。排序时，先将整个序列调整为大顶堆，然后把堆顶元素与当前堆的最后一个元素交换，这样最大值就被放到了序列末尾；接着缩小堆的范围，对剩余元素重新调整为大顶堆，再次交换堆顶元素和末尾元素。如此反复进行，直到堆中只剩一个元素为止，整个序列就变成了有序序列。堆排序的时间复杂度在最好、平均和最坏情况下都为 $O(n \log n)$ ，空间复杂度为 $O(1)$ ，因此它的效率比较稳定，而且不需要额外开辟大量辅助空间；不过它在排序过程中元素跳跃交换较多，所以它属于**不稳定排序算法**。

● 堆排序伪代码

Algorithm HeapSort

Input: Array a consisting of n elements

Output: a will be sorted in nondecreasing order.

```
function HEAPSORT( $a, n$ )  
  for  $i \leftarrow n/2$  downto  $0$  do
```

```

        HEAPADJUST(a, i, n)
    end for

    for i ← 1 to n - 1 do
        swap a[0] and a[n - i]
        HEAPADJUST(a, 0, n - i)
    end for
end function

function HEAPADJUST(a, parent, len)
    while 2 * parent + 1 < len do
        child ← 2 * parent + 1
        if child + 1 < len and a[child] < a[child + 1] then
            child++
        end if

        if a[child] > a[parent] then
            swap a[parent] and a[child]
            parent ← child
        else
            break
        end if
    end while
end function

```

- 堆排序实现 TOP-k

堆排序通常分两步：**第一步：建堆**把 n 个元素构造成一个堆，复杂度为： $O(n)$ ；**第二步：反复取堆顶并调整堆**取出堆顶后，需要重新调整堆。一共有 $n - 1$ 次取堆顶操作，每次调整堆的复杂度为：

$$O(\log n)$$

因此这一部分复杂度为：

$$O(n \log n)$$

所以堆排序总复杂度为：

$$O(n + n \log n) = O(n \log n)$$

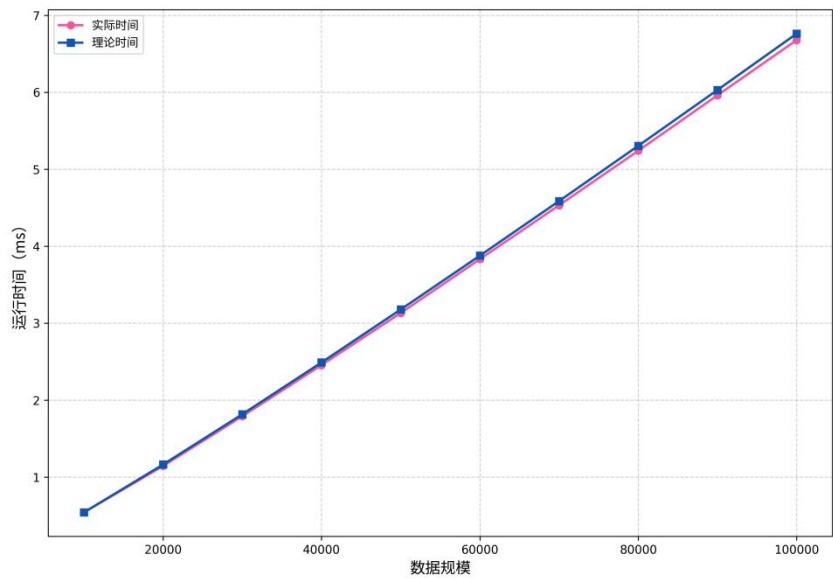
即：最好情况： $O(n \log n)$ ，平均情况： $O(n \log n)$ 最坏情况： $O(n \log n)$

- 堆排序实验结果

本实验采用了不同规模的数据作为输入，并统一保存在数据文件中，以保证各算法测试条件一致。我们通过编写 C++ 脚本测量了算法在不同数据规模下的实际运行时间，同时根据时间复杂度公式推导出理论用时。最后，将实验结果整理成一张表格 [/workspace/hanyu_workspace/result/heap_compare_results.csv](#)，并绘制了实际时间与理论时间的对比曲线图 [/workspace/hanyu_workspace/result/heap_compare.png](#)，用于分析算法理论效率与实际表现之间的关系。

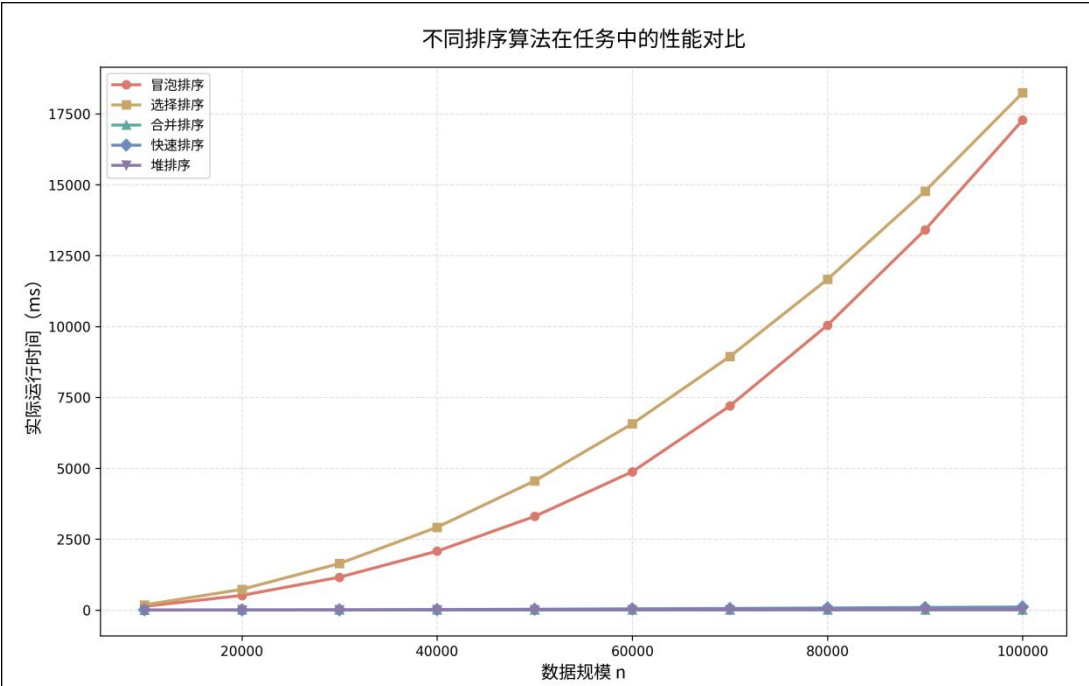
表 5 堆排序实验结果

n	actual_time_ms	theoretical_time_ms
10000	0.541095	0.541095
20000	1.144757	1.163633
30000	1.790608	1.816912
40000	2.456329	2.490153
50000	3.132583	3.178238
60000	3.832750	3.878152
70000	4.532963	4.587904
80000	5.241865	5.306077
90000	5.962648	6.031613
100000	6.680232	6.763690



图表 15 堆排序理论时间与实际时间对比

五种排序算法的对比



图表 16 各排序实际时间对比

三、优化算法

(一) 只排序前 10 轮

采用冒泡排序或选择排序算法。由于这两种算法具有一个独特的性质：**每一轮排序都能确定一个元素的最终位置**。基于这一性质，在求解 TOP- k 问题时，不必对全部数据完成完整排序，只需进行 k 轮排序，就可以得到排好序的前 k 个数据，从而减少不必要的计算开销，提高算法效率。

弊端：这种方法虽然避免了完整排序，但其本质仍然是基于冒泡排序或选择排序的逐轮比较，因此当数据规模 n 很大时，整体效率仍然不高。对于冒泡排序和选择排序而言，执行前 k 轮的时间复杂度约为 $O(nk)$ 。当 k 较小时，这种方法还有一定可行性；但如果 n 很大，即使只排序前 k 轮，也仍然需要进行大量比较操作，运行时间会明显增加。因此，这种方法更适用于 k 较小、数据规模不太大的场景，在大规模数据下仍然存在效率瓶颈

实验结果：

此为采用冒泡排序的实验结果

表 6 合并排序实验结果

n	actual_time_ms	theoretical_time_ms
10000	0.532251	0.532251
20000	1.063317	1.064501
30000	1.593578	1.596752
40000	2.125298	2.129003
50000	2.660560	2.661253
60000	3.187676	3.193504
70000	3.720219	3.725755
80000	4.252893	4.258006

90000	4.780330	4.790256
100000	5.316752	5.322507

(二) 使用改良版快速排序

其基本思路是：先从当前数据中选取一个枢轴值（pivot），再通过一次划分操作将数据分为两部分，即大于枢轴的元素和小于枢轴的元素。由于实验目标只是找出前 (K) 个最大的数，而不是对全部数据完成完整排序，因此每一轮划分后，只需保留较大的一部分继续处理，而较小的一部分可以直接舍弃。随着划分不断进行，待处理数据规模会快速缩小，直到剩余元素数量接近 (K) 时，再对这部分数据进行排序并输出前 (K) 个结果。相比于直接对全部 (n) 个数据进行完整快速排序，该方法减少了大量无关数据的排序开销，更适合在 (n) 很大而 (K) 很小的情况下求解 Top-K 问题，具有较高的实际运行效率。

该算法每轮通过一次划分操作缩小待处理数据范围，在 pivot 选取较合理时，待处理规模大致按比例递减，因此平均时间复杂度为 $O(n)$ ；当最后剩余元素数量接近 K 时，再进行局部排序，故总平均时间复杂度可表示为 $O(n + K\log K)$ 。在最坏情况下，若 pivot 选择不佳，时间复杂度会退化为 $O(n^2)$ 。

“合理”的 pivot 就是能够使划分结果尽量平衡、从而快速缩小问题规模的枢轴值。实际实现中，通常不必精确求真正的中位数，而可以采用随机选取、三数取中等方法来提高 pivot 选取的合理性。

表 7 改良版快速排序实验结果

n	avg_time_ms
10000	0.041537
20000	0.090879
30000	0.156768
40000	0.193903
50000	0.265618
60000	0.272163
70000	0.345645
80000	0.406897
90000	0.304261
100000	0.474994

四、不同 k 值的影响

对于 TOP-k 问题，当 n 固定时，k 的增大会直接影响排序或筛选阶段的计算量，但影响程度取决于所采用的算法：

1. 冒泡排序、选择排序这类“做 k 轮即可确定前 k 个元素”的算法

这类算法不需要完整排完全部 n 个元素，只需执行前 k 轮。

因此时间复杂度可写成：

$$O(nk)$$

当 n=10000 固定时，运行时间会随着 k 的增大而近似线性增长。

也就是说：k 很小时（如 k=10），只需少量轮次，运行较快；k 逐渐增大时，比较次数会明显增加；当 k 接近 n 时，就退化得接近完整排序，效率明显下降。所以这类算法对 k 非常敏感。

2. 插入排序用于 TOP-k 的情况

如果仍按“只维护前 k 个有序结果”的思路实现，那么它的复杂度通常约为：

$$O(nk)$$

因此在 $n=10000$ 不变时， k 增大同样会导致运行时间明显增长，而且增长趋势通常也接近线性。

3. 合并排序、快速排序这类先整体排序再取前 k 个的算法

如果算法流程是“先把全部数据排好序，再取前 k 个”，那么复杂度主要由完整排序决定：

合并排序：

$$O(n \log n)$$

快速排序（平均）：

$$O(n \log n)$$

这时 k 只影响最后“截取前 k 个结果”的操作，代价通常只是：

$$O(k)$$

相较于前面的整体排序开销很小，因此在 $n=10000$ 固定时， k 的变化对总运行时间影响不明显。

换句话说， k 从 10 增加到 1000，时间可能只会有轻微变化，曲线整体较平缓。

4. 堆排序或堆维护 TOP-k 的算法

如果采用大小为 k 的堆维护当前前 k 个元素，则复杂度一般为：

$$O(n \log k)$$

当 $n=10000$ 固定时，随着 k 增大， $\log k$ 也会增大，因此运行时间会增加，但增长速度比 $O(nk)$ 慢得多。

也就是说： k 增大时，效率会下降；但下降幅度比较平缓；相比冒泡、选择这类算法，对 k 的敏感度更低。

五、稀疏向量

一个用户理论上会有很多维兴趣特征，但事实上真正的非零兴趣项会很少。好比一个平台上可以选择电影、音乐、旅游、编程、美妆或军事等等兴趣维度。但一个用户真正有兴趣的项目可能只有很少几个，所以大部分位置都是 0。这时如果仍然按长度为 n 的完整数组逐项去算，会浪费很多时间，因为大量计算其实都是： $0 \times 0, 0 \times x, x \times 0$ ，这对余弦相似度里面的点积没有贡献。

● 稀疏向量的存放

我们可以采用“类+数组”的形式存放。

设置“兴趣”类：

```
class Interest {  
public:
```

```
int index;
double weight;
};
```

然后

```
Interest user[s];
```

或者是动态数组存储

```
vector<Interest> user;
```

这种方法的内存开销通常是比较可控的。以一个兴趣项为例，若其中包含一个 `int index` 和一个 `double weight`，理论上分别占 4 字节和 8 字节，共 12 字节。但由于 C++ 中通常存在内存对齐机制，编译器往往会将其补齐到 16 字节，因此一个兴趣项实际可近似看作占用 16 Byte。

在这种情况下，如果一个用户只有 s 个非零兴趣，那么该用户的稀疏向量存储空间大约为 $16s$ Byte。比如当 $s = 10$ 时，大约占 160 Byte；当 $s = 100$ 时，大约占 1600 Byte，约为 1.56 KB；当 $s = 1000$ 时，大约占 16000 Byte，约为 15.6 KB。由此可以看出，在非零项数量不是特别大的情况下，单个用户所占用的内存其实并不高。

如果进一步考虑多个用户，假设共有 m 个用户，且每个用户平均有 s 个非零兴趣项，那么总存储空间大约为 $16ms$ Byte。例如，当用户数为 10000、每个用户平均有 10 个非零兴趣项时，总内存约为 1.53 MB；当用户数为 100000、每个用户平均有 10 个非零兴趣项时，总内存约为 15.3 MB；当用户数增加到 100000 且每个用户平均有 100 个非零兴趣项时，总内存约为 152.6 MB。虽然此时内存占用已经比较明显，但相较于直接存储完整的稠密向量，仍然节省了大量空间。

可以把稀疏向量计算余弦相似度的复杂度分成三部分来讲：点积复杂度、模长复杂度、总复杂度。

● 余弦相似度计算

设两个用户的兴趣向量分别为 A 和 B ，余弦相似度公式为：

$$\cos(A, B) = \frac{A \cdot B}{\|A\| \|B\|}$$

其中：

$$A \cdot B = \sum_{i=1}^n A_i B_i$$

$$\|A\| = \sqrt{\sum_{i=1}^n A_i^2}, \|B\| = \sqrt{\sum_{i=1}^n B_i^2}$$

如果是普通稠密向量，需要把全部 n 个维度都扫描一遍，所以复杂度是 $O(n)$ 。但对于稀疏向量，大多数维度都是 0，因此没有必要再遍历全部维度，只需要处理非零项即可。

假设向量 A 有 s_1 个非零项，向量 B 有 s_2 个非零项，并且使用“按维度编号升序存储的二元组数组”来表示稀疏向量，例如：

$$A = [(i_1, a_1), (i_2, a_2), \dots, (i_{s_1}, a_{s_1})]$$

$$B = [(j_1, b_1), (j_2, b_2), \dots, (j_{s_2}, b_{s_2})]$$

那么模长的计算只需要对各自的非零项平方求和即可，因此：

$\|A\|$ 的计算复杂度为 $O(s_1)$

$\|B\|$ 的计算复杂度为 $O(s_2)$

而点积计算时，只需要找出两个向量中“下标相同”的非零项。若采用双指针法同时扫描两个有序数组，每个非零项最多被访问一次，因此点积复杂度为：

$$O(s_1 + s_2)$$

所以，整个余弦相似度计算的总复杂度为：

$$O(s_1 + s_2)$$

如果进一步假设两个用户的非零兴趣数大致相同，即

$$s_1 \approx s_2 \approx s$$

那么总复杂度可以近似写成：

$$O(s)$$

这说明稀疏向量的余弦相似度计算复杂度，不再取决于总维度 n ，而主要取决于非零项个数 s 。当

$$s \ll n$$

时，稀疏表示会比稠密表示高效得多。

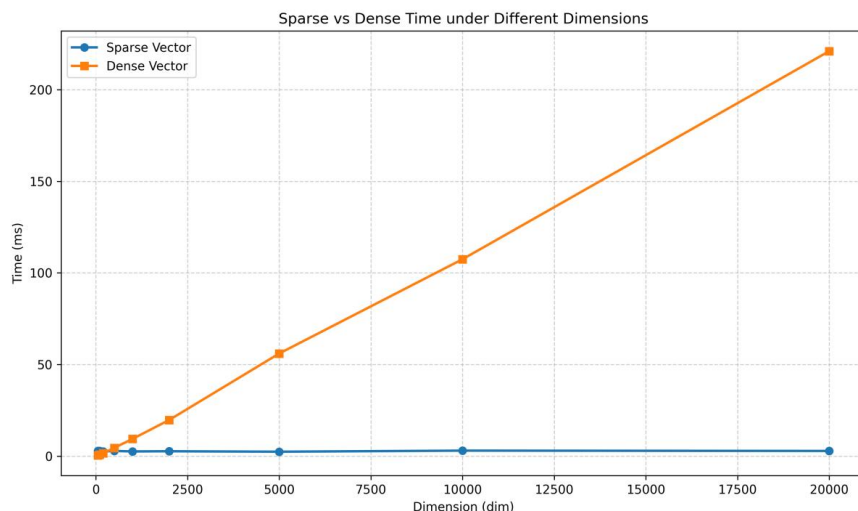
● 不同维度下的效率

实验结果表明，在本实验设置下（兴趣向量维度固定为 50，每个稀疏向量仅有 10 个非零项），稀疏向量计算余弦相似度的实际耗时反而高于稠密向量。其原因在于，虽然从理论复杂度上看，稀疏向量的计算复杂度可由 $O(\text{DIM})$ 降低为 $O(s)$ ，但本实验中的维度 $\text{DIM} = 50$ 较小，稠密向量仅需进行 50 次连续乘加运算，且内存连续、分支较少、易于被编译器优化，因此实际常数开销较低。相比之下，稀疏向量虽然只处理非零项，但需要额外进行下标比较、分支跳转和结构体访问，这些开销在低维场景下抵消了其理论优势，因此导致实际运行时间高于稠密向量。

因此额外增加了维度更大情况下的比较。

表 8 合并排序实验结果

dim	n	sparse_time_ms	dense_time_ms
50	20000	2.83546	0.359054
100	20000	2.84034	0.766675
200	20000	2.51324	1.50865
500	20000	2.79542	4.50046
1000	20000	2.53062	9.37709
2000	20000	2.62238	19.7243
5000	20000	2.37458	55.9989
10000	20000	2.9736	107.468
20000	20000	2.79752	221.046



图表 18 稀疏向量与稠密向量计算余弦相似度用时对比

● 不同 s 下的效率

稠密时间基本恒定是正常的

稠密向量每次都要做的是：第一个用户和其余 **9999** 个用户比较，每次比较都要扫完整个 **10000** 维向量

所以总工作量大致固定为：

$$9999 \times 10000$$

稠密向量的计算量和 s 没关系，因此 `dense_time_ms` 基本保持不变是正常的。始终是：

$$64.607451 \text{ ms}$$

2 稀疏时间随 s 增大而增大，这也正常

稀疏向量的余弦相似度计算，主要代价和非零项数 s 有关。

当 s 变大时：每个向量里非零项更多，点积时双指针或稀疏乘法处理的数据更多所以运行时间会逐步上升：

$s = 5$ 时：0.19 ms

$s = 10$ 时：0.25 ms

$s = 20$ 时：0.35 ms

$s = 50$ 时：0.72 ms

$s = 100$ 时：1.65 ms

$s = 200$ 时：3.48 ms

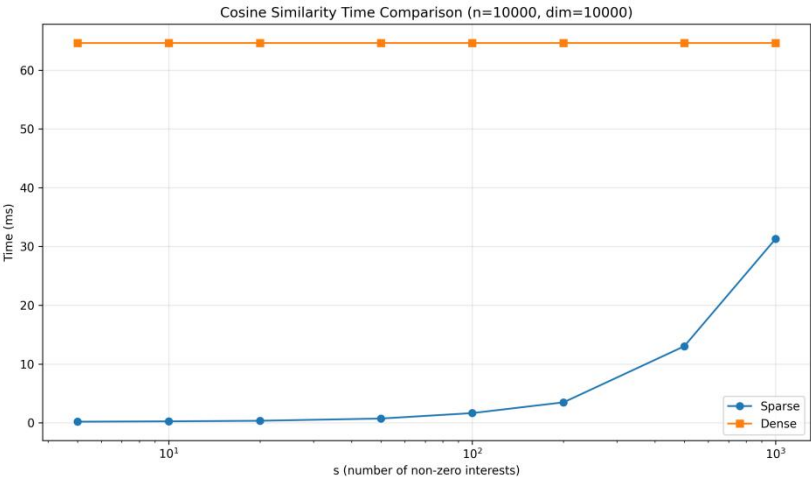
$s = 500$ 时：13.03 ms

$s = 1000$ 时：31.30 ms

表 9 合并排序实验结果

dim	s	sparse_time_ms	dense_time_ms
10000	5	0.1913662999868393	64.60745073854923
	10	0.2474561333656311	64.60745073854923

10000	20	0.34628001352151233	64.60745073854923
10000	50	0.7232061276833216	64.60745073854923
10000	100	1.647941768169403	64.60745073854923
10000	200	3.480698292454084	64.60745073854923
10000	500	13.025171433885893	64.60745073854923
10000	1000	31.299086287617683	64.60745073854923



图表 19 稀疏度不同情况下稀疏向量与稠密向量的用时对比

五、实验收获

通过本次实验可以得到的主要结论是：在实际问题中，算法设计不能只停留在理论复杂度层面，而应紧密结合任务目标、数据规模和数据特征进行针对性优化。

对于本实验中的 TOP-k 问题，真正需要的是“找出前 k 个最大值”，而不是对全部数据完成完整排序，因此直接套用完整排序算法往往会产生大量无效计算。

实验表明，若只关注前 k 个结果，则可以利用部分排序或选择型算法减少不必要的处理开销，例如冒泡排序、选择排序可以只执行前 k 轮，快速排序可以改造成快速选择，只递归处理可能包含前 k 个元素的一侧，从而使算法更贴合问题本身。进一步分析说明，不同算法虽然在理论上都可完成任务，但在实际运行中表现差异明显：当数据规模较小时，简单算法实现直接、常数开销小，仍具有一定可用性。

对于稀疏向量与稠密向量的比较，实验进一步表明，数据表示方式本身就是算法优化的重要组成部分：当特征向量具有明显稀疏性时，采用稀疏存储与稀疏计算能够显著减少无意义的乘加操作；而当维度和稀疏度变化时，两种方法的效率优势也会发生转移，这说明算法效率不仅取决于“用什么算法”，还取决于“如何组织数据”。因此，本实验最终得到的核心认识是：解决实际问题时，应先抓住任务目标，识别真正的计算瓶颈，再根据数据规模、k 值大小、向量稀疏性以及工程实现代价选择合适的算法和数据结构，

而不是机械地追求某一种通用排序方法。

指导教师批阅意见：

成绩评定：

指导教师签字：

2024 年 4 月 5 日

备注：

- 注：1、报告内的项目或内容设置，可根据实际情况加以调整和补充。
2、教师批改学生实验报告时间应在学生提交实验报告时间后 10 日内。